

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Естественно-языковой интерфейс ИС»

Тема: «Разработка системы машинного перевода документов»

Выполнил:

Студент 4 курса

Группы ИИ-23

Романюк А. П.

Проверил:

Булей Е. В.

Ход работы

Цель работы – освоить на практике основные принципы машинного перевода документов.

- ✓ на входе – естественно-языковой текст на входном языке, подлежащий процедуре машинного перевода;
- ✓ подсчитать количество слов во входном тексте, количество переведенных слов, определить грамматическую информацию (теги частей речи и их расшифровка, для этого следует использовать функциональность лр. №3 весенний семестр).
- ✓ на выходе:
 - перевод входного текста на выходной язык;

Код программы:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext, messagebox, ttk
from deep_translator import GoogleTranslator
import threading
import nltk
import spacy

nlp = spacy.load("en_core_web_sm")

class TXTTranslatorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("TXT Переводчик")
        self.root.geometry("1000x800")

        self.file_content = None
        self.stop_translation_flag = False

        self.create_widgets()

    def create_widgets(self):
        control_frame = tk.Frame(self.root)
        control_frame.pack(fill=tk.X, padx=10, pady=10)

        self.open_button = tk.Button(control_frame, text="Открыть TXT", command=self.open_txt, font=("Arial", 12))
        self.open_button.pack(side=tk.LEFT, padx=5)

        self.translate_button = tk.Button(control_frame, text="Перевести", command=self.start_translation_thread,
                                          font=("Arial", 12))
        self.translate_button.pack(side=tk.LEFT, padx=5)

        self.stop_button = tk.Button(control_frame, text="Стор", command=self.stop_translation, state=tk.DISABLED,
                                     font=("Arial", 12))
        self.stop_button.pack(side=tk.LEFT, padx=5)

        self.language_var = tk.StringVar(value="en")
        language_label = tk.Label(control_frame, text="Язык перевода:", font=("Arial", 12))
        language_label.pack(side=tk.LEFT, padx=5)
        language_menu = ttk.Combobox(control_frame, textvariable=self.language_var, values=["en", "de"],
                                     font=("Arial", 12), state="readonly")
        language_menu.pack(side=tk.LEFT, padx=5)

        text_frame = tk.Frame(self.root)
        text_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        source_frame = tk.LabelFrame(text_frame, text="Исходный текст", font=("Arial", 12))
        source_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=5)

        self.text_area = scrolledtext.ScrolledText(source_frame, wrap=tk.WORD, width=40, height=20, font=("Arial", 12))
        self.text_area.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        translated_frame = tk.LabelFrame(text_frame, text="Переведенный текст", font=("Arial", 12))
        translated_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=5)

        self.translated_text_area = scrolledtext.ScrolledText(translated_frame, wrap=tk.WORD, width=40, height=20,
```

```

        font=("Arial", 12))
self.translated_text_area.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

analysis_frame = tk.Frame(self.root)
analysis_frame.pack(fill=tk.X, padx=10, pady=10)

def open_txt(self):
    file_path = filedialog.askopenfilename(filetypes=[("Text Files", "*.txt")])
    if file_path:
        try:
            with open(file_path, 'r', encoding='utf-8') as file:
                self.file_content = file.read()

            self.text_area.delete(1.0, tk.END)
            self.text_area.insert(tk.INSERT, self.file_content)

            self.translated_text_area.delete(1.0, tk.END)
            self.translate_button.config(state=tk.NORMAL)
        except Exception as e:
            messagebox.showerror("Ошибка", f"Не удалось открыть TXT: {e}")

def start_translation_thread(self):
    self.stop_translation_flag = False
    translation_thread = threading.Thread(target=self.translate_text)
    translation_thread.start()
    self.stop_button.config(state=tk.NORMAL)
    self.translate_button.config(state=tk.DISABLED)
    self.open_button.config(state=tk.DISABLED)

def stop_translation(self):
    self.stop_translation_flag = True
    self.stop_button.config(state=tk.DISABLED)

def translate_text(self):
    if self.file_content:
        try:
            self.translated_text_area.delete(1.0, tk.END)
            self.translate_button.config(text="Переводим...")

            translator = GoogleTranslator(source='en', target=self.language_var.get())
            lines = self.file_content.splitlines()
            for line in lines:
                if self.stop_translation_flag:
                    break

                if line.strip():
                    try:
                        translated = translator.translate(line.strip())
                    except Exception as e:
                        translated = f"Ошибка перевода строки: {e}"
                    self.translated_text_area.insert(tk.END, translated + '\n\n')
                    self.translated_text_area.update_idletasks()
                    self.translate_button.config(state=tk.NORMAL, text="Перевести")
                    self.open_button.config(state=tk.NORMAL)
                    self.stop_button.config(state=tk.DISABLED)
            except Exception as e:
                messagebox.showerror("Ошибка", f"Не удалось перевести текст: {e}")
                self.translate_button.config(state=tk.NORMAL)
                self.stop_button.config(state=tk.DISABLED)
                self.open_button.config(state=tk.NORMAL)
        else:
            messagebox.showwarning("Внимание", "Сначала откройте TXT файл.")

def count_words(self):
    if self.file_content:
        word_count = len(self.file_content.split())
        messagebox.showinfo("Подсчет слов", f"Количество слов в тексте: {word_count}")
    else:
        messagebox.showwarning("Внимание", "Сначала откройте TXT файл.")

def lexical_analysis(self):
    if self.file_content:
        try:
            sentences = nltk.sent_tokenize(self.file_content)

```

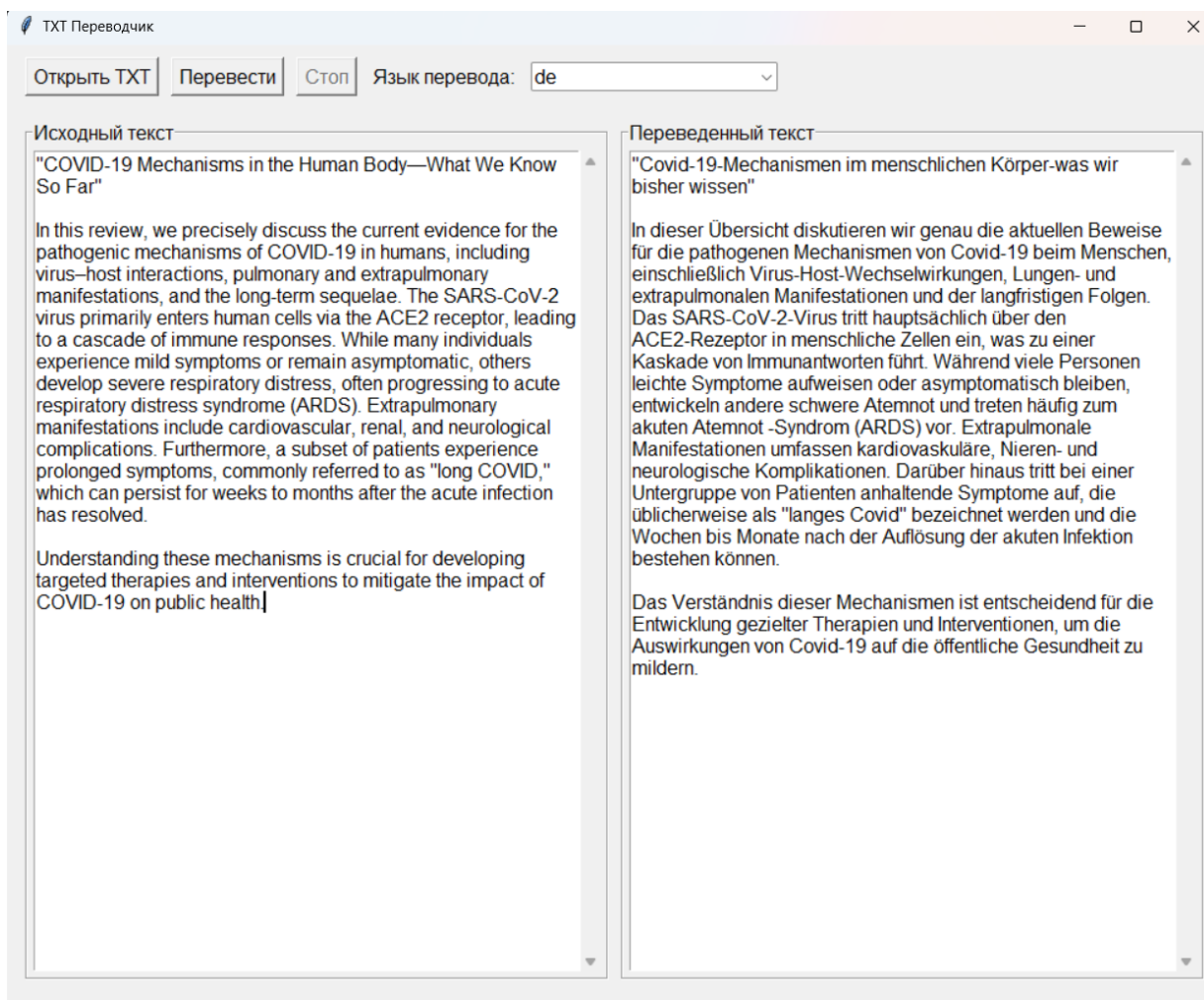
```

if sentences:
    first_sentence = sentences[0]
else:
    messagebox.showwarning("Внимание", "Нет предложений для разбора.")
    return

tokens = nltk.word_tokenize(first_sentence)
tagged = nltk.pos_tag(tokens)

lexical_result = "\n".join([f"{word}: {tag}" for word, tag in tagged])
messagebox.showinfo("Лексический разбор", f"Лексический разбор первого предложения:\n{lexical_result}")
except Exception as e:
    messagebox.showerror("Ошибка", f"Не удалось выполнить лексический разбор: {e}")
else:
    messagebox.showwarning("Внимание", "Сначала откройте TXT файл.")
if __name__ == "__main__":
    root = tk.Tk()
    app = TXTTranslatorApp(root)
    root.mainloop()

```



Вывод: в ходе выполнения лабораторной работы освоил принципы разработки системы машинного перевода документов